CBL · CONTEXT BUDGET LANGUAGE

代码审阅指南

上下文预算语言 · 答辩前的模块阅读与问答准备

每人一章：先读什么、为什么这样设计、怎么亲手验证、可能被问什么、局限在哪。代码摘录与链接全部固定在提交 50b4276，与仓库逐行一致；文中每条命令都实际跑过。

徐同帅
词法分析 · 语法分析

舒若凡
语义检查 · 中间表示 · 负责人

杨晨
预算执行引擎

祝梦倩
审计报告 · 网页演示 · 测试

仓库 github.com/rovan666/cbl · 提交 50b4276 · 测试 54/54 通过

2026 年 10 月 26 日

目录

> 写给四位组员：答辩时每人都要能讲清自己负责的模块——**做了什么、为什么这样做、局限在哪**。每一节按"阅读顺序 → 设计决定 → 动手验证 → 可能被问的问题 → 已知局限"组织。文中代码链接都固定在提交 `50b4276`，行号不会随后续修改错位。"动手验证"里的每条命令和现象都实际跑过；改完代码记得 `git checkout .` 还原。

0 所有人先看（15 分钟） 3

1 徐同帅：词法分析与语法分析 5

1.1 阅读顺序 5
1.2 设计决定（答辩要讲得出理由） 5
1.3 核心代码导读 5
1.4 动手验证 7
1.5 可能被问的问题 8
1.6 已知局限 8

2 舒若凡：语义检查与中间表示（负责人） 9

2.1 阅读顺序 9
2.2 设计决定 9
2.3 核心代码导读 9
2.4 动手验证 11
2.5 可能被问的问题 11
2.6 已知局限 12

3 杨晨：预算执行引擎 13

3.1 阅读顺序 13
3.2 设计决定 13
3.3 核心代码导读 14
3.4 动手验证 17
3.5 可能被问的问题 17
3.6 已知局限 17

4 祝梦倩：审计报告、网页演示与测试 18

4.1 阅读顺序 18
4.2 设计决定 19
4.3 核心代码导读 19
4.4 动手验证 21
4.5 可能被问的问题 21
4.6 已知局限 21

5 任何人都可能被问的问题 22

6 在 GitHub 上审阅时的清单 23

附录 A 术语表 24

附录 B 命令速查 25

附录 C 五分钟演示脚本 26

第 0 章

所有人先看（15 分钟）

一条数据从头走到尾——每一步的产物都是下一步的输入：

```
.cbl 源码 ──lexer──▶ Token 列表 ──parser──▶ AST ──semantic──▶ 诊断列表 ──ir──▶ JSON 组装计划
                                                                                  │
                                  上下文材料 + 本轮提问 + 可用额度 B ──────────────▶ engine
                                                                                  │
                                                  最终 Prompt + 审计记录 ◀──┘ ──report/webui──▶ 文本 / HTML / 网页
```

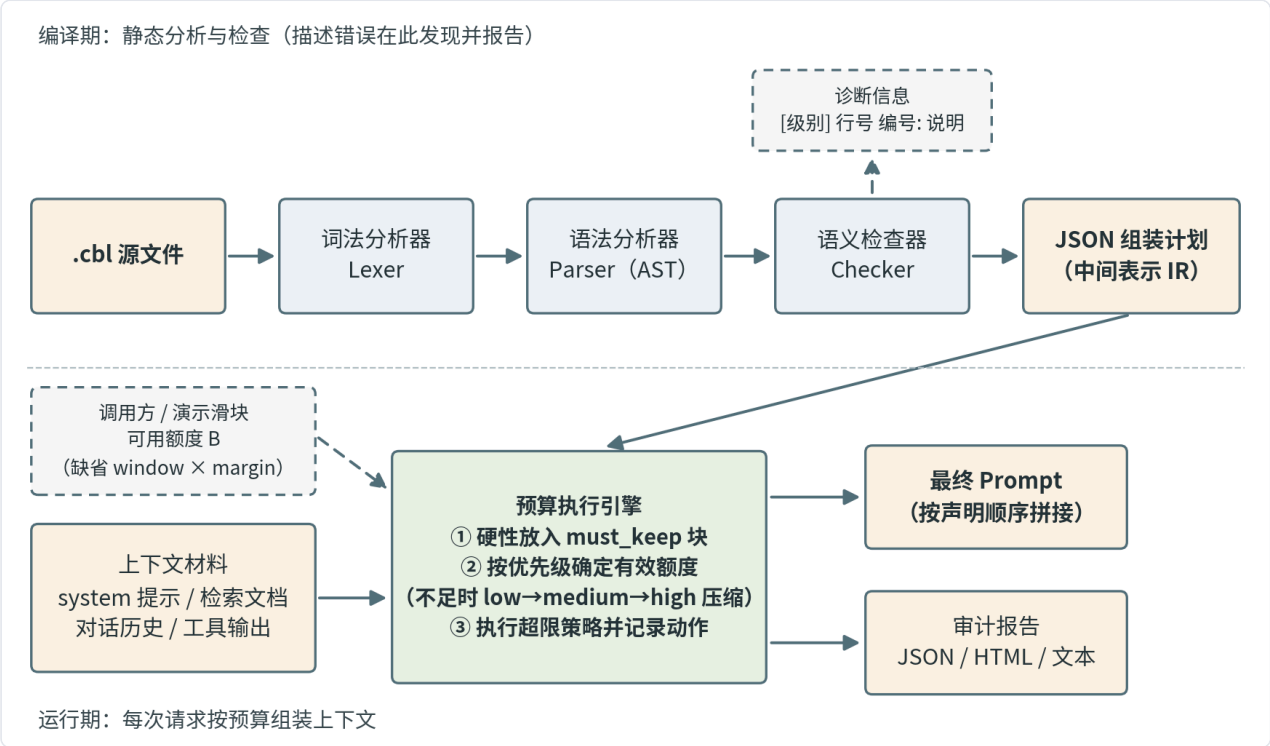


图 0-1　CBL 系统总体框架：上半部分为编译期，下半部分为运行期

阶段	产物的数据结构	定义在哪
词法	`Token(kind, value, line, col)`	tokens.py
语法	`Program` / `WindowDecl` / `BlockDecl` / `RuleDecl`	ast_nodes.py
语义	`Diagnostic(level, line, msg, code)`	semantic.py#L51
IR	普通 dict（可直接 `json.dumps`）	ir.py#L19
组装	`(prompt: str, audit: dict)`	engine.py#L385

先跑起来（在 `cbl/` 目录）：

```
python tests/run_tests.py                              # 54 个用例应全部通过
python -m cbl check examples/medical_assistant.cbl     # 看 JSON 组装计划
python -m cbl run   examples/medical_assistant.cbl     # 看审计报告；再试 --budget 6000、--budget 1700
python -m cbl serve                                    # 预算实验台（或双击 启动预算实验台.bat）
```

在 GitHub 上提意见：打开任一提交（如 `7a5398c`），鼠标移到代码行左侧点 `+` 就能对那一行评论；整体意见可以开 Issue。需要直接推代码的话，让舒若凡在仓库 Settings → Collaborators 里加你。

第 1 章

徐同帅

词法分析与语法分析

负责文件

`tokens.py · lexer.py · parser.py · ast_nodes.py`

三个核心概念

1. 手写确定有限自动机做词法：7 个分支、最长匹配
2. 递归下降、LL(1)：一个产生式一个函数，看一个 Token 定分支
3. 终结符匹配同时检查类型；报错精确到行号、期望符号、实际读到什么

必须能脱稿回答

1. 为什么手写，不用 ANTLR?
2. 文法为什么是 LL(1)?
3. 能一次报出多个语法错误吗?

1.1 阅读顺序

1. tokens.py：21 个关键字、界符表。**先记住 Token 只有四个字段**。
2. lexer.py `tokenize`：一个 `while` 循环里按首字符分 7 个分支—— 空白、 注释、 字符串、 数字、 标识符/关键字、 双字符运算符、 单字符界符。
3. parser.py 基础工具：`_is` / `expect` / `expect_kind` ——所有产生式都靠这三个函数"吃" Token。
4. 产生式函数，**对照 EBNF 读**（文件开头的注释就是文法）： `parse_program` → `parse_window` → `parse_block` → `parse_rule` → `parse_operand` → `_choice` 。

1.2 设计决定（答辩要讲得出理由）

- **手写递归下降，不用 ANTLR**：文法只有十条左右产生式；每个产生式对应一个函数，报错能精确到"第几行、期望什么、实际读到什么"，比生成器的默认报错更适合教学（计划书问题一）。
- **文法是 LL(1) 的**：每个选择点只看下一个 Token 就能决定走哪条分支，不需要回溯——顶层看 `window/block/rule` ，块内看条目关键字， `operand` 看是 `INT` 还是 `window` ， `INT` 后面再看有没有 `%` 。文法里没有左递归。
- **最长匹配**：数字分支读完整数后，只有"点后面还是数字"才继续读成 FLOAT； `<=` 必须先于 `<` 判断（双字符分支在单字符之前）。
- **终结符匹配同时检查类型**（ `_is` ）：这是在 `50b4276` 修掉的真实漏洞——之前只比较值，于是 `window 8000 ";"` 里的字符串 `";"` 被当成分号、 `"cap": 100` 被当成条目都能通过。**这是一个很好的答辩素材**：说明"值相同 ≠ 同一个终结符"，词法类别本身就是语法的一部分。
- **报错写清楚读到的是什么**：字符串会标明"字符串"；缺分号报在**缺的那一行**（上一个 Token 所在行），并注明实际读到的 Token 在哪行。
- **缺省值在语法分析时填入**： `priority=medium` 、 `overflow=truncate_tail` ，但 must_keep 块未写 overflow 时缺省为 `error` （parser.py#L144，配合语义约束⑨）。

1.3 核心代码导读

以下代码取自提交 `50b4276` ，左侧是文件里的真实行号。先读每段下面的说明，再看代码。

cbl/lexer.py 第 67–94 行 在 GitHub 打开

```
        # 4) 数字：整数 INT；带小数点且点后有数字的是 FLOAT（如 0.9）
        if ch.isdigit():
            buf = []
            while i < n and text[i].isdigit():
                buf.append(text[i])
                advance()
            kind = "INT"
            if peek() == "." and peek(1).isdigit():
                buf.append(".")
                advance()
                while i < n and text[i].isdigit():
                    buf.append(text[i])
                    advance()
                kind = "FLOAT"
            tokens.append(Token(kind, "".join(buf), start_line, start_col))
            continue

        # 5) 标识符 / 关键字：ident = [A-Za-z_][A-Za-z0-9_]*
        #    只认 ASCII：str.isalpha() 会把汉字也当字母，与正规式不符
        if (ch.isascii() and ch.isalpha()) or ch == "_":
            buf = []
            while i < n and ((text[i].isascii() and text[i].isalnum()) or text[i] == "_"):
                buf.append(text[i])
                advance()
            word = "".join(buf)
            kind = "KEYWORD" if word in KEYWORDS else "IDENT"
            tokens.append(Token(kind, word, start_line, start_col))
            continue
```

最长匹配：读完整数后，只有“点后面还是数字”才继续读成 FLOAT。**标识符只认 ASCII**（`isascii()`），读完整个单词再查关键字表，决定是 KEYWORD 还是 IDENT——这就是“关键字是保留字”在代码里的样子。

cbl/parser.py 第 58–69 行 在 GitHub 打开

```
    @staticmethod
    def _is(tok: Token, value: str) -> bool:
        """终结符匹配：值相同且必须是关键字或界符——字符串 ";"、"cap" 不能冒充终结符。"""
        return tok.value == value and tok.kind in ("KEYWORD", "SYMBOL")

    def expect(self, value: str, what: str = "") -> Token:
        """断言下一个 Token 是指定终结符，否则报错。what 用于拼错误信息。"""
        tok = self.peek()
        if not self._is(tok, value):
            line = self.tokens[self.pos - 1].line if value == ";" and self.pos > 0 else None
            raise ParseError(f"期望 '{value}'（{what or '语法要求'}）", tok, line)
        return self.advance()
```

所有产生式都靠 `expect` 消耗终结符。`_is` 要求 Token 的**类型**是关键字或界符——字符串 `";"` 值相同也不能冒充分号（提交 50b4276 修复的漏洞）。缺分号时把报错行改为上一个 Token 所在行。

cbl/parser.py 第 112–147 行 在 GitHub 打开

```python
    def parse_block(self) -> BlockDecl:
        kw = self.expect("block", "块声明开头")
        name = self.expect_kind("IDENT", "块名（标识符）")
        blk = BlockDecl(name=name.value, line=kw.line)
        self.expect("{", "块体开始")
        seen = set()
        while not self._is(self.peek(), "}"):
            item = self.peek()
            if item.kind != "KEYWORD" or item.value not in ("source", "cap", "priority", "overflow", "keep_recent"):
                raise ParseError("块内只接受 source/cap/priority/overflow/keep_recent 条目", item)
            if item.value in seen:
                raise ParseError(f"条目 '{item.value}' 在同一个块里重复声明", item)
            seen.add(item.value)
            key = item.value
            self.advance()
            self.expect(":", f"{key} 条目")
            if key == "source":
                t = self.peek()
                if t.kind not in ("STRING", "IDENT"):
                    raise ParseError("source 只接受字符串或内置来源标识符", t)
                blk.source_kind = "string" if t.kind == "STRING" else "ident"
                blk.source = self.advance().value
            elif key == "cap":
                blk.cap = int(self.expect_kind("INT", "额度整数").value)
            elif key == "priority":
                blk.priority = self._choice(PRIORITIES, "优先级")
            elif key == "overflow":
                blk.overflow = self._choice(OVERFLOWS, "超限策略")
            elif key == "keep_recent":
                blk.keep_recent = int(self.expect_kind("INT", "保留条数整数").value)
            self.expect(";", f"{key} 条目结尾")
        self.expect("}", "块体结束")
        # 缺省值约定：must_keep 块未声明 overflow 时缺省为 error（死保信息不裁剪）
        if "overflow" not in seen and blk.priority == "must_keep":
            blk.overflow = "error"
        return blk
```

一个产生式 = 一个函数：块体 `{ blockItem }` 写成 while 循环，循环里看**一个**条目关键字决定走哪条分支（LL(1)）。条目重复、类型不对都在这里报错；最后为 must_keep 块填入缺省 overflow = error。

1.4 动手验证

```
printf 'window 8000 ;\nblock a { source: "a.txt" ; cap: 100 ; priority: "high" ; }\n' > t1.cbl
python -m cbl check t1.cbl
# 语法错误 行2: 优先级必须是 must_keep / high / medium / low 之一，实际读到 字符串 "high"

printf 'window 8000\nblock a { source: "a.txt" ; cap: 100 ; }\n' > t2.cbl
python -m cbl check t2.cbl
# 语法错误 行1: 期望 ';'（window 声明结尾），实际读到 'block'（行2）

printf 'window 8000 ;\nblock 历史 { source: "a.txt" ; cap: 100 ; }\n' > t3.cbl
python -m cbl check t3.cbl
# 词法错误 行2 列7: 无法识别的字符 '历'    ← 标识符只认 ASCII，与正规式一致
```

再试一个：把 parser.py#L61 的 `and tok.kind in (...)` 删掉，跑测试，看 `case_strings_cannot_impersonate_terminals` 怎么失败。

1.5 可能被问的问题

问题	回答要点
为什么不用 ANTLR / yacc?	文法小；手写能控制报错质量；课程要求锻炼前端能力。代价是改文法要手改函数。
你的文法是 LL(1) 吗？怎么说明?	是。逐个选择点说明：各分支的 FIRST 集互不相交（见上面"设计决定"第二条），所以看一个 Token 就能定分支；没有左递归。
词法分析器对应什么自动机?	手写的确定有限自动机：循环开头按当前字符跳到某个"状态"（7 个分支），在状态内累积字符直到不能再接受为止（最长匹配），然后产出 Token。
能一次报出多个语法错误吗?	不能，遇到第一个语法错误就停（抛异常）。可以做"恐慌模式"恢复：出错后跳到下一个 `;` 或 `}` 继续。**语义错误则一次全部报出**（见第 2 节）。
`"high"` 和 `high` 有什么区别?	前者是 STRING 类 Token，后者是 KEYWORD 类；枚举值必须是关键字。讲一下 `50b4276` 那个修复。
关键字能当块名吗?	不能，21 个关键字都是保留字，例如 `block error { ... }` 会报"期望块名（标识符）"。

1.6 已知局限

- 字符串不支持转义、不能跨行（按计划书正规式的规定）。
- 没有语法错误恢复，一次只报第一个语法错误。
- 关键字保留，块名不能叫 `error`、`high` 等。

第 2 章

舒若凡

语义检查与中间表示（负责人）

负责文件

`semantic.py` · `ir.py` · `tests/run_tests.py`（PAIRS）

三个核心概念

1. 错误目录 16 项 + 表驱动检查表，新增约束只需三步
2. 属性文法：Σcap 是综合属性，window/margin 起继承属性作用
3. 编译期断言与运行期断言的划分；IR 显式写出全部缺省值

必须能脱稿回答

1. 属性文法体现在哪?
2. 九条约束怎么保证实现正确?
3. 为什么要 JSON 中间表示?

2.1 阅读顺序

1. `CATALOG` 错误目录：16 项，每项 = (所属约束, 级别, 说明模板)。**先读懂这张表，后面的代码都是在实现它。**
2. `_diag`：按编号造一条诊断。
3. 九个检查函数，一约束一函数：①②③④⑤⑥⑦⑧⑨
4. `CHECKS` 检查表与 `check()` 主流程：只有 3 行，所有检查跑一遍、按行号排序。
5. ir.py `to_ir`：AST → JSON。
6. 测试：`PAIRS` 正反例表、`case_catalog_covered`（目录里每一项都必须有正反例）。

2.2 设计决定

- **错误目录 + 表驱动**（计划书问题四）：新增一条检查只需三步——目录里加一行、写一个函数挂进 `CHECKS`、`PAIRS` 里加一对正反例。漏写测试会被 `case_catalog_covered` 拦下。
- **属性文法体现在哪**：`total_reserved`（各块 cap 之和）是**综合属性**，由各 block 的 cap 自底向上汇总到 Program；`window`、`margin` 从根节点向下传给各项检查使用，起**继承属性**的作用。约束①②⑦都建立在这些属性之上。
- **一次报出全部语义错误**：九个检查都跑完再汇总，按行号排序——用户一次能改完，不用"改一个、编译一次"。
- **编译期 vs 运行期断言**：只依赖 cap 与 window 的断言（`total_reserved`）编译期就能求值；依赖运行期数据的（`every_block_min_tokens`，即压缩后的有效额度）留给引擎。失败时都按规则自己的 `on_violation` 定级别（R702 的级别因此不固定）。
- **约束③为什么在语法已经拦住后还要查**：语法分析器保证源码里的枚举合法，但 AST 也可能被程序直接构造（测试、以后的网页编辑器），语义层再复核一次，防御性检查。
- **约束⑨（must_keep 的 overflow 只能是 error）**：从语言层面兑现"死保信息不被任何策略裁剪"的承诺。
- **IR 里缺省值全部显式写出**：引擎不需要知道 CBL 的缺省规则；`source_kind` 告诉引擎是读文件还是取内置来源；`metric` / `op` 单独给出，引擎不必再解析断言字符串。

2.3 核心代码导读

以下代码取自提交 `50b4276`，左侧是文件里的真实行号。先读每段下面的说明，再看代码。

cbl/semantic.py 第 28–48 行 在 GitHub 打开

```
CATALOG = {
    "E101": ("①", "错误", "各块额度之和 {total} 超过上限 window×{margin}={limit}"),
    "E102": ("①", "错误", "margin {margin} 应在 (0, 1] 区间内"),
    "E201": ("②", "错误", "must_keep 块额度之和 {mk} 超过窗口 {win}，预算无解"),
    "E301": ("③", "错误", "块 '{name}' 的 priority 取值 '{value}' 不合法，应为 {options} 之一"),
    "E302": ("③", "错误", "块 '{name}' 的 overflow 取值 '{value}' 不合法，应为 {options} 之一"),
    "E303": ("③", "错误", "规则 '{name}' 的 on_violation 取值 '{value}' 不合法，应为 {options} 之一"),
    "E401": ("④", "错误", "名字 '{name}' 重复定义（首次出现于 行{first}）"),
    "W501": ("⑤", "警告", "块 '{name}' 声明了 keep_recent，但 overflow 为 {overflow}："
                          "keep_recent 只对 summarize / sliding_window 有意义，将被忽略"),
    "E502": ("⑤", "错误", "块 '{name}' 的 keep_recent 必须为正整数（实际为 {value}）"),
    "E601": ("⑥", "错误", "块 '{name}' 的来源 '{source}' 不是内置来源标识符"
                          "（可用：{options}）；文件路径请用双引号括起"),
    "E701": ("⑦", "错误", "规则 '{name}' 的度量 '{metric}' 未定义（可用：{options}）"),
    "R702": ("⑦", None, "规则 '{name}' 断言不成立：{metric}={actual}，要求 {op} {bound}（{expr}）"),
    "E801": ("⑧", "错误", "块 '{name}' 缺少必填项 source"),
    "E802": ("⑧", "错误", "块 '{name}' 缺少必填项 cap（或 cap 不是正整数）"),
    "E803": ("⑧", "错误", "window 容量必须为正整数（实际为 {win}）"),
    "E901": ("⑨", "错误", "块 '{name}' 为 must_keep，overflow 只能是 error（实际为 {overflow}），"
                          "死保信息不允许被任何策略裁剪"),
}
```

错误目录：16 项，每项 = (所属约束, 级别, 说明模板)。级别为 None 的 R702 由规则自己的 on_violation 决定。后面所有检查函数都只是在“按编号造诊断”。

cbl/semantic.py 第 145–163 行 在 GitHub 打开

```
def check_assert(prog):
    """⑦ 断言合法性：度量必须预定义；静态断言（只涉及 cap 与 window）在编译期求值，
    失败按 on_violation 报错误或警告；动态断言留给运行期引擎求值。"""
    win = prog.window.tokens
    total = sum(b.cap for b in prog.blocks)
    for r in prog.rules:
        if r.metric not in BUILTIN_METRICS:
            yield _diag("E701", r.line, name=r.name, metric=r.metric,
                        options=_options(BUILTIN_METRICS))
            continue
        if r.metric not in STATIC_METRICS:
            continue
        bound = _operand_value(r.operand, win)
        ok = total <= bound if r.op == "<=" else total >= bound
        if not ok:
            level = "错误" if r.on_violation == "error" else "警告"
            yield _diag("R702", r.line, level=level, name=r.name, metric=r.metric,
                        actual=total, op=r.op, bound=f"{bound:g}",
                        expr=_fmt_operand(r.operand))
```

静态断言编译期求值：度量先查是否预定义（E701）；只依赖 cap 与 window 的 total_reserved 当场算出 Σcap 与右值比较，其余度量留给运行期。Σcap 正是 AST 上自底向上汇总的**综合属性**。

cbl/semantic.py 第 184–192 行 在 GitHub 打开

```
184 CHECKS = [check_total, check_unsolvable, check_enums, check_unique,
185           check_keep_recent, check_source, check_assert, check_required,
186           check_protection]
187
188
189 def check(prog: Program) -> list:
190     """返回诊断列表（按行号排序）；含 '错误' 级诊断则视为编译失败。"""
191     diags = [d for fn in CHECKS for d in fn(prog)]
192     return sorted(diags, key=lambda d: d.line)
```

表驱动主流程：九个检查函数排成一张表，全部跑完再按行号排序——一次报出所有语义错误。

tests/run_tests.py 第 234–247 行 在 GitHub 打开

```
234 def _pair_case(code, pos, neg):
235     def fn():
236         d_pos = _diags_of(pos)
237         assert not has_error(d_pos), f"正例不应有错误：{d_pos}"
238         assert all(d.code != code for d in d_pos), f"正例不应报 {code}：{d_pos}"
239         d_neg = _diags_of(neg)
240         assert any(d.code == code for d in d_neg), f"反例应报 {code}：{d_neg}"
241     return fn
242
243
244 def case_catalog_covered():
245     # 错误目录里的每一项都必须有正反例，防止新增检查漏写测试
246     missing = set(CATALOG) - {code for code, _, _ in PAIRS}
247     assert not missing, f"以下编号缺少正反例：{sorted(missing)}"
```

正反例 + 覆盖率自检：正例必须编译通过且不含该编号，反例必须报出该编号；`case_catalog_covered` 保证目录里每一项都配了测试。

2.4 动手验证

1. 把 semantic.py#L184 的 `CHECKS` 里删掉 `check_source`，跑测试——会有 3 个用例失败，其中 `pair_E601` 报"反例应报 E601：[]"。**测试能抓住"检查被删"，说明它真的在把关。**
2. 练习：试着新增一条约束 **W1001「cap 小于 64 给警告」**。按"三步"做完后，`case_catalog_covered` 会提醒你必须补正反例。做一遍就能讲清楚表驱动的好处。

2.5 可能被问的问题

问题	回答要点
属性文法体现在哪?	综合属性 Σcap 自底向上，继承属性 window/margin 自顶向下；约束是定义在属性上的谓词。
为什么不在语法分析时顺便检查?	分层：语法管"句子结构对不对"，语义管"意思说不说得通"。比如 Σcap 超窗，必须等所有 block 都读完才能判断。
九条约束怎么保证实现正确?	每个错误编号一对正反例 + 覆盖率自检 + 变异检查（删掉检查函数，测试必须失败）。
R702 为什么有时是错误、有时是警告?	规则级别由用户写的 `on_violation` 决定；内置约束①是固定的错误，规则机制让用户写出比它更严格（或更宽松提示）的纪律。

问题	回答要点
为什么要 JSON 中间表示，不直接解释 AST?	编译器与引擎解耦，可以各自替换；JSON 可存档、可 diff、任何语言都能读。
符号表怎么做的?	block 与 rule 共用一张表（一个 dict），CBL 没有嵌套作用域，一层就够。

2.6 已知局限

- E301~E303 在正常源码中触发不到（语法层已拦截），只防程序化构造的 AST。
- 没有检查"某块 cap 本身就小于 `every_block_min_tokens` 阈值"这种静态即可发现的矛盾——是一个可以补充的约束。
- 符号表只有一层（够用，因为语言没有嵌套）。

第 3 章

杨晨

预算执行引擎

负责文件

`engine.py` · `examples/materials/` · `tests/engine_cases.py`

三个核心概念

1. 三步走：定可用额度 B → 逐级压缩有效额度 → 执行五种策略
2. 最大余数法、保守 token 估算 + 二分截断
3. BM25（中文二字组合）与规则式摘要；计数器、摘要器可插拔

必须能脱稿回答

1. 额度为什么不回流?
2. 为什么先把 low 压到 0?
3. token 估算不准怎么办?

3.1 阅读顺序

1. `assemble_parts` 主流程：**先通读这一个函数**，它就是计划书的"三步走"。
2. 第 2 步额度分配：`allocate` → `_proportional`（最大余数法）。
3. 第 3 步五种策略，统一签名的策略表：truncate_tail、sliding_window、summarize、rerank_drop、error。
4. 基础设施：`estimate_tokens` → `_fit_prefix`（二分） → `_soft_cut`；`bm25_scores`、`rule_summary`；`eval_rules`；`load_material`（含路径沙箱）。

3.2 设计决定

- **可用额度 B 与压缩次序**（计划书运行期语义 ⓑ）：B 缺省为 window×margin，可调小；Σcap 超过 B 时按 low → medium → high **逐级**压缩，一级压到 0 才动下一级；must_keep 不参与压缩。
- **最大余数法**：同级按 cap 等比例分配时先取整数部分，剩下的名额按小数部分从大到小补，保证各块之和**恰好**等于目标值，不多不少（例：3 块各 100 分 200 → 67、67、66）。
- **策略统一签名、放进表里**：主流程只写 `STRATEGIES[b["overflow"]](...)`，新增策略不改主流程——和语义检查的表驱动是同一种思路。
- **token 保守估算**（ⓔ）：中文每字 1.5、英文每 3 字符 1 个，宁多勿少——估多了只是少放一点，估少了会真的超窗。估算函数对任意前缀**单调不减**，所以截断时可以二分查找"最长能放下的前缀"。
- **计数器、摘要器可插拔**：`assemble(counter=..., summarizer=...)`，可以换成 tiktoken 或大模型摘要。
- **截断尽量停在句末**（`_soft_cut`：在前缀最后 20% 里找句号）。
- **rerank_drop = BM25 + 中文二字组合**：零依赖、不需要分词器。试过再加单字作检索词，结果无关段落得分反而升高（噪声大于收益），已否决。
- **拒绝组装带审计记录**（`AssemblyError.audit`）：报告能说清楚是哪一步、为什么失败。
- **字符串来源限制在材料目录内**：网页允许编辑源码，不限制的话 `source: "../../任意文件"` 能读出电脑上任意文件。



图 3-1　逐级压缩：B 从 7200 调到 6000，超出的 1100 先由 low 级的 tool_output 承担（500→0，整块舍弃），剩下 600 再由 medium 级的 chat_history 承担（1600→1000）；high 与 must_keep 不受影响。

3.3　核心代码导读

以下代码取自提交 `50b4276`，左侧是文件里的真实行号。先读每段下面的说明，再看代码。

cbl/engine.py　第 178–208 行　在 GitHub 打开

```
178  def allocate(blocks, budget):
179      """返回 {块名: 有效额度}；must_keep 总额超过 budget 时抛 ValueError。"""
180      eff = {b["name"]: b["cap"] for b in blocks}
181      must = sum(b["cap"] for b in blocks if b["priority"] == "must_keep")
182      if must > budget:
183          raise ValueError(f"must_keep 块额度之和 {must} 超过本次可用额度 {budget}")
184      excess = sum(eff.values()) - budget
185      for level in SQUEEZE_ORDER:
186          if excess <= 0:
187              break
188          group = [b for b in blocks if b["priority"] == level]
189          total = sum(b["cap"] for b in group)
190          if total == 0:
191              continue
192          cut = min(excess, total)
193          _proportional(group, total - cut, eff)
194          excess -= cut
195      return eff
196
197
198  def _proportional(group, keep, eff):
199      """同级按 cap 等比例分配 keep；最大余数法取整，保证总和恰为 keep（同余数按声明顺序）。"""
200      total = sum(b["cap"] for b in group)
201      exact = [b["cap"] * keep / total for b in group]
202      floors = [math.floor(x) for x in exact]
203      rest = keep - sum(floors)
204      order = sorted(range(len(group)), key=lambda i: -(exact[i] - floors[i]))
205      for i in order[:rest]:
206          floors[i] += 1
207      for b, v in zip(group, floors):
208          eff[b["name"]] = v
```

第 2 步 • 有效额度：先确认 must_keep 放得下 B，再按 low → medium → high 逐级削减超出部分。`_proportional` 用**最大余数法**取整：先发整数部分，剩余名额按小数部分从大到小补，总和恰好等于目标。

cbl/engine.py 第 46–78 行 在 GitHub 打开

```python
def estimate_tokens(text: str) -> int:
    """保守 token 估算，宁多勿少：
    - 中日韩等全角字符：每字 1.5（主流分词器约 0.7~1.5，取上沿）
    - 连续英文/数字：每 3 字符 1 个（英文平均约 4 字符/token，取偏多）
    - 其他符号与换行各 1 个；空格不计（通常并入相邻词）
    对任意前缀单调不减，truncate 系列策略依赖这一点做二分查找。
    需要精确计数时，可给 assemble(counter=...) 传入真实分词器（如 tiktoken）。"""
    total, wide, i, n = 0, 0, 0, len(text)
    while i < n:
        m = _WORD.match(text, i)
        if m:
            total += math.ceil(len(m.group()) / 3)
            i = m.end()
            continue
        ch = text[i]
        if unicodedata.east_asian_width(ch) in "WF":
            wide += 1
        elif ch == "\n" or not ch.isspace():
            total += 1
        i += 1
    return total + math.ceil(wide * 1.5)


def _fit_prefix(text, limit, counter):
    """不超过 limit 的最长前缀长度（二分查找）。"""
    lo, hi = 0, len(text)
    while lo < hi:
        mid = (lo + hi + 1) // 2
        if counter(text[:mid]) <= limit:
            lo = mid
        else:
            hi = mid - 1
    return lo
```

保守估算 + 二分：中文每字 1.5、英文每 3 字符 1 个，宁多勿少。估算对任意前缀单调不减，所以 `_fit_prefix` 能二分出“最长能放下的前缀”。

cbl/engine.py 第 256–273 行 在 GitHub 打开

```python
def rerank_drop(units, sep, limit, block, ctx, counter, summarizer):
    """以本轮提问为查询做 BM25 打分，按分数从低到高整段丢弃，剩余段落保持原顺序。"""
    if len(units) == 1:
        return truncate_tail(units, sep, limit, block, ctx, counter, summarizer)
    scores = bm25_scores(units, ctx.query)
    drop_order = sorted(range(len(units)), key=lambda i: (scores[i], -i))   # 同分先丢靠后的
    alive = set(range(len(units)))
    for i in drop_order:
        if counter(sep.join(units[j] for j in sorted(alive))) <= limit or len(alive) == 1:
            break
        alive.discard(i)
    kept = [units[j] for j in sorted(alive)]
    dropped = len(units) - len(kept)
    how = "相关度最低的" if ctx.query else "（无查询，按位置）末尾的"
    if counter(sep.join(kept)) > limit:              # 只剩一段仍超额：截断它
        out, detail = truncate_tail(kept, sep, limit, block, ctx, counter, summarizer)
        return out, f"丢弃{how} {dropped} 段，剩余一段再{detail}"
    return sep.join(kept), f"丢弃{how} {dropped} 段"
```

相关度丢弃：以本轮提问为查询算 BM25，分数从低到高整段丢弃，剩余段落保持原顺序；只剩一段仍超额时退化为截断。没有提问时按位置从末尾丢。

cbl/engine.py 第 391–450 行 在 GitHub 打开

```python
def assemble_parts(plan: dict, ctx: Context, counter=estimate_tokens, summarizer=rule_summary):
    """同 assemble，但返回 ([(块名, 该块最终文本), ...], 审计记录)，按声明顺序；
    被舍弃或来源缺失的块文本为空串。网页演示据此逐块展示"AI 实际看到了什么"。"""
    window = plan["window"]
    budget = ctx.budget if ctx.budget is not None else int(window * plan["margin"])
    audit = {"window": window, "budget": budget, "query": ctx.query, "status": "ok",
             "error": None, "total_used": 0, "blocks": [], "rules": []}

    def reject(msg):
        audit["status"], audit["error"] = "rejected", msg
        audit["total_used"] = sum(e["used"] for e in audit["blocks"])
        raise AssemblyError(msg, audit)

    try:
        eff = allocate(plan["blocks"], budget)                      # 第 2 步
    except ValueError as e:
        reject(str(e))

    parts = []
    for b in plan["blocks"]:                                        # 第 3 步
        entry = {"name": b["name"], "priority": b["priority"], "cap": b["cap"],
                 "effective": eff[b["name"]], "original": 0, "used": 0,
                 "action": "完整保留", "detail": ""}
        audit["blocks"].append(entry)
        try:
            mat = load_material(b, ctx)
        except MaterialMissing as e:
            if b["priority"] == "must_keep":
                entry.update(action="来源缺失", detail=str(e))
                reject(f"must_keep 块 '{b['name']}' 来源缺失（{e}），无法保证死保信息")
            entry.update(action="来源缺失", detail=f"{e}，按空内容处理")
            parts.append((b["name"], ""))
            continue
        units, sep, _ = mat
        text = sep.join(units)
        entry["original"] = counter(text)
        limit = eff[b["name"]]
        if limit == 0:
            entry["action"] = "整块舍弃"
            entry["detail"] = "有效额度被压缩为 0"
            parts.append((b["name"], ""))
            continue
        if entry["original"] > limit:
            try:
                text, detail = STRATEGIES[b["overflow"]](units, sep, limit, b, ctx,
                                                         counter, summarizer)
            except _OverflowRejected as e:
                entry.update(action="error", detail="超额，拒绝组装")
                reject(str(e))
            entry.update(action=b["overflow"], detail=detail)
        entry["used"] = counter(text)
        parts.append((b["name"], text))

    audit["total_used"] = sum(e["used"] for e in audit["blocks"])
    audit["rules"] = eval_rules(plan, eff)
    for r in audit["rules"]:
        if not r["passed"] and r["level"] == "error":
            reject(f"规则 '{r['name']}' 不成立（{r['metric']}={r['actual']} {r['op']} "
                   f"{r['bound']:g} 为假），on_violation 为 error，拒绝组装")
    return parts, audit
```

主流程（三步走）：定 B → 分配有效额度 → 逐块读材料、超额执行策略；最后求值运行期断言。任何一步要拒绝组装都走 `reject()`，把截至当时的审计记录一并带出。

3.4 动手验证

```
python -m cbl run examples/medical_assistant.cbl                # 缺省 B=7200：四种策略都触发
python -m cbl run examples/medical_assistant.cbl --budget 6000  # tool_output 压到 0、chat_history 压到 1000
python -m cbl run examples/medical_assistant.cbl --budget 1700  # 低于死保总额 1800：拒绝组装
```

1. 把 engine.py#L21 的压缩次序改成 `("high", "medium", "low")`，跑测试——5 个用例失败（单元、端到端、快照、网页、HTTP 接口各层都拦住了）。
2. **换计数器**，体会"计数器不同，裁剪决策就不同"：

```
from cbl.cli import compile_file
from cbl.engine import Context, assemble
ir, _ = compile_file("examples/medical_assistant.cbl")
_, a = assemble(ir, Context.from_dir("examples/materials"), counter=len)   # 按字符数计
print([(b["name"], b["original"], b["used"], b["action"]) for b in a["blocks"]])
# chat_history 用 len 算是 1559 ≤ 1600，于是"完整保留"不再摘要；
# tool_output 是英文 JSON，用 len 算反而更大（976）。—这就是为什么要保守估算。
```

3.5 可能被问的问题

问题	回答要点
未用满的额度为什么不回流给别的块?	可预测、可审计：每块拿多少只取决于声明，不取决于别的块这次材料多不多（ⓑ）。代价是可能有浪费，由用户调 cap 解决。
为什么 low 压到 0 才压 medium，而不是大家按比例一起压?	优先级的语义就是"严格先后"；按比例压会让高优先级也受损。担心低优先级被饿死，就写 `no_starvation` 规则兜底——示例里 B=6000 时它会报警告。
token 估算不准怎么办?	故意偏多，保证"引擎认为放得下，真实分词器下也放得下"；需要精确就传 `counter`。编译期只看 cap 数字，与分词器无关。
rerank_drop 的局限?	字面匹配，认不出同义词：提问说"发烧"、资料写"发热"就对不上，示例里"咽拭子检测"一段就被误丢。改进：同义词表或向量检索（策略可插拔）。
摘要是怎么做的?	规则式：每轮用户/助手各取首个有信息量的句子（≥8 字），再补与提问共享 ≥2 个检索词的句子；超长时停在句末。可换成大模型摘要。
性能如何？拖滑块为什么不卡?	示例一次编译 + 组装约 12 ms（只编译前端约 0.2 ms）；截断用二分，复杂度 O(n log n)。
must_keep 的内容超过 cap 怎么办?	拒绝组装并报错（ⓐ），绝不静默裁剪——约束⑨保证它的策略必为 error。

3.6 已知局限

- token 是估算值，不是真实分词结果。
- rerank 只做字面匹配；规则式摘要质量有限。
- rerank 以段落为单位，只有剩一段仍超额时才截断它。
- 块之间的分隔符 `\n\n` 不计入额度（开销极小，由 margin 预留覆盖）。

第 4 章

祝梦倩

审计报告、网页演示与测试

负责文件

`report.py` · `webui.py` · `static/`（`report.js` · `report.css` · `demo.html`） · `tests/webui_cases.py`

三个核心概念

1. 审计记录 JSON 为主，文本 / HTML / 网页是它的视图
2. 网页与 HTML 报告共用一个渲染器；两道关防 XSS
3. 快照测试防止裁剪行为被悄悄改变；请求序号防止画面乱序

必须能脱稿回答

1. 怎么防止材料里的恶意内容注入页面?
2. 快照测试和单元测试有什么区别?
3. 局域网模式安全吗?

4.1 阅读顺序

1. report.py `render_text`：文本报告，格式与计划书第 5 节一致。
2. `_embed_json` 与 `build_html`：独立 HTML 报告怎么把数据安全地塞进页面。
3. webui.py `analyze`（一次编译 + 组装，返回页面要的全部数据）→ `_unit_status`（每段/每轮的状态）→ `make_handler`（HTTP 接口）。
4. static/report.js：`rows` → `renderWindowBar` → `renderBlocks` → `renderStatic`。static/demo.html 里的内联脚本负责滑块、编辑器和请求。
5. 测试：快照 `_snapshot`、webui_cases.py（含在随机端口起真实服务的 `case_server_endpoints`）。

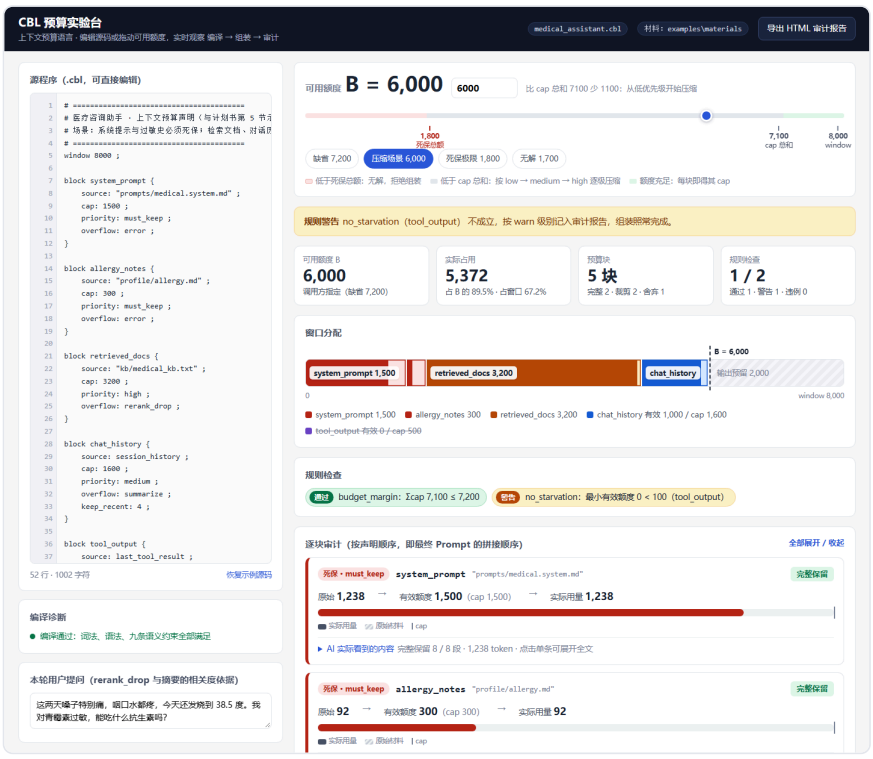


图 4-1 预算实验台：可用额度调到 6000 时，tool_output 被整块舍弃、chat_history 被压缩

4.2 设计决定

- **审计记录以 JSON 为主格式**，文本、HTML、网页都只是它的不同视图。
- **网页和 HTML 报告共用一个渲染器**（report.js）：两边视觉永远一致，只维护一份代码；HTML 报告把样式、渲染器、数据全部内嵌，单文件离线可看。
- **防 XSS 两道关**：渲染器只用 DOM API 和 `textContent` 写文本，从不拼接 HTML 字符串（材料内容不可信）；内嵌的 JSON 把 `< > &` 转成 `\u` 形式，材料里即使有 `</script>` 也截断不了脚本。
- **单位状态怎么判定**：看这段/这轮原文是否出现在最终文本里（保留）、只出现首尾片段（部分保留）、位于保留原文之前且有摘要（并入摘要），否则丢弃。**曾经的 bug**：摘要会引用早期轮次的首句，这些轮次被误判成"部分保留"，页面显示"保留 7/8 轮"，实际只保留了 4 轮——修法是"并入摘要"的判断必须先于"部分保留"，并加了回归测试。
- **拖动不乱序**：每次请求带序号，只认最新响应，旧请求用 `AbortController` 取消。
- **快照测试**（计划书问题五 ③）：把审计报告文本存进 `tests/snapshots/`，每次逐字比对——引擎有人改了裁剪行为会立刻暴露。
- **演示服务的安全**：缺省只监听 127.0.0.1；静态文件名不能含路径；请求体超过 256KB 只看请求头就返回 413。

4.3 核心代码导读

以下代码取自提交 `50b4276`，左侧是文件里的真实行号。先读每段下面的说明，再看代码。

cbl/webui.py 第 39–59 行 在 GitHub 打开

```
39  def _unit_status(units, final, action):
40      """给每个裁剪单位标状态：kept / partial / summarized / dropped。"""
41      if action == "完整保留":
42          return ["kept"] * len(units)
43      if action in ("整块舍弃", "来源缺失", "error"):
44          return ["dropped"] * len(units)
45      has_summary = final.startswith(SUMMARY_TAG)
46      kept = [u in final for u in units]
47      first_kept = next((i for i, k in enumerate(kept) if k), len(units))
48      out = []
49      for i, u in enumerate(units):
50          if kept[i]:
51              out.append("kept")
52          elif has_summary and i < first_kept:
53              # 摘要覆盖保留原文之前的单位；须先于"部分保留"判断——摘要会引用这些单位的首句
54              out.append("summarized")
55          elif u[:PARTIAL_PROBE] in final or u[-PARTIAL_PROBE:] in final:
56              out.append("partial")       # truncate 截掉尾巴、sliding_window 只留末尾
57          else:
58              out.append("dropped")
59      return out
```

单位状态判定：原文出现在最终文本里 = 保留；位于保留原文之前且有摘要 = 并入摘要；只出现首尾片段 = 部分保留。注意“并入摘要”必须先于“部分保留”判断——摘要会引用这些轮次的首句（曾经的 bug）。

cbl/static/report.js 第 13–32 行 在 GitHub 打开

```
13  function h(tag, attrs) {
14    var el = document.createElement(tag);
15    if (attrs) {
16      for (var k in attrs) {
17        if (!Object.prototype.hasOwnProperty.call(attrs, k) || attrs[k] == null) continue;
18        if (k === "text") el.textContent = attrs[k];
19        else if (k === "cls") el.className = attrs[k];
20        else if (k === "style") el.setAttribute("style", attrs[k]);
21        else if (k.slice(0, 2) === "on") el.addEventListener(k.slice(2), attrs[k]);
22        else el.setAttribute(k, attrs[k]);
23      }
24    }
25    for (var i = 2; i < arguments.length; i++) {
26      var c = arguments[i];
27      if (c == null || c === false) continue;
28      if (Array.isArray(c)) c.forEach(function (x) { if (x != null) el.appendChild(typeof x === "string" ? document.cre
29      else el.appendChild(typeof c === "string" ? document.createTextNode(c) : c);
30    }
31    return el;
32  }
```

防 XSS 第一道关：渲染器只用 `createElement` 与 `textContent` 构建节点，从不把材料内容拼进 HTML 字符串。

cbl/report.py 第 78–81 行 在 GitHub 打开

```
78  def _embed_json(obj):
79      """可安全放进 <script> 的 JSON：< > & 转义为 \\u 形式，材料里即使有 </script> 也截断不了脚本。"""
80      s = json.dumps(obj, ensure_ascii=False)
81      return s.replace("<", "\\u003c").replace(">", "\\u003e").replace("&", "\\u0026")
```

防 XSS 第二道关：HTML 报告内嵌的 JSON 把 < > & 转成 \u 形式，材料里即使有 `</script>` 也截断不了脚本；JSON 解析后原文不变。

cbl/static/demo.html 第 276–288 行 在 GitHub 打开

```
276   function analyze() {
277     var my = ++state.seq;
278     if (state.ctrl) state.ctrl.abort();
279     state.ctrl = new AbortController();
280     $("busy").classList.add("on");
281     $("view").classList.add("stale");
282     fetch("/api/analyze", { method: "POST", headers: { "Content-Type": "application/json" },
283                             body: payload(), signal: state.ctrl.signal })
284       .then(function (r) { return r.json(); })
285       .then(function (data) { if (my === state.seq) render(data); })
286       .catch(function (e) { if (e.name !== "AbortError") toast("请求失败：" + e.message); })
287       .finally(function () { if (my === state.seq) { $("busy").classList.remove("on"); $("view").classList.remove("sta
288   }
```

拖动不乱序：每次请求序号 +1，只渲染序号仍是最新的响应；旧请求用 AbortController 取消。

tests/engine_cases.py 第 179–188 行 在 GitHub 打开

```python
def _snapshot(name, text):
    path = os.path.join(SNAPSHOTS, name)
    if os.environ.get("CBL_UPDATE_SNAPSHOTS") == "1" or not os.path.exists(path):
        os.makedirs(SNAPSHOTS, exist_ok=True)
        with open(path, "w", encoding="utf-8", newline="\n") as f:
            f.write(text + "\n")
        return
    want = open(path, encoding="utf-8").read().rstrip("\n")
    assert text == want, (f"审计报告与快照 {name} 不一致——若是预期变化，"
                          f"设 CBL_UPDATE_SNAPSHOTS=1 重跑以更新。\n实际：\n{text}")
```

快照测试：首次运行生成快照，之后逐字比对；预期内的变化用 CBL_UPDATE_SNAPSHOTS=1 接受。

4.4 动手验证

1. 在 report.py#L56 的 `if` 下面加一行 `act = act.replace("完整保留", "完整")`，跑测试—— `case_snapshot_reports` 失败并打印实际输出。如果这个改动是你想要的，执行 `CBL_UPDATE_SNAPSHOTS=1 python tests/run_tests.py` 接受新快照（PowerShell 里先 `$env:CBL_UPDATE_SNAPSHOTS=1` ）。
2. `python -m cbl serve` ，地址后加 `?budget=6000` 、 `?budget=1700` 看三种场景；改坏源码看诊断和行号标红；点"导出 HTML 审计报告"。
3. XSS 实验：在"本轮用户提问"里输入 `</script><script>alert(1)</script>` ，导出报告再打开——不会弹窗，提问原样显示在页眉。

4.5 可能被问的问题

问题	回答要点
为什么用 JS 渲染，不用 Python 直接拼 HTML?	网页要实时交互，本来就得用 JS 渲染；让 HTML 报告复用同一个渲染器，就不用维护两份视觉代码。
怎么防止材料里的恶意内容注入页面?	`textContent` + JSON 转义两道关；有专门的测试（ `case_html_report_embed_safe` ）。
快照测试和普通单元测试有什么区别?	单元测试断言具体性质；快照测试"冻结"整个输出，任何变化都要人确认，适合防止行为被悄悄改变。
拖滑块会不会卡、会不会显示错乱?	一次请求约 12 ms；序号 + 取消旧请求，保证画面只显示最新结果。
局域网模式安全吗?	同一 Wi-Fi 的人都能访问、能编辑源码；但来源路径被限制在材料目录内，读不到其他文件。它定位是演示工具，没有登录鉴权。

4.6 已知局限

- 单位状态是基于子串的启发式判断，材料里如果有完全重复的段落可能误判。
- 演示服务没有鉴权（ `--lan` 时同网段可访问）。
- HTML 报告需要开启 JavaScript 才能显示（原始数据仍在页面源码里）。

第 5 章

任何人都可能被问的问题

问题	回答要点
CBL 和直接在 Python 里写配置有什么区别?	静态检查（编译期发现超窗、死保无解、拼写错误）、可审计（每次组装有报告）、与模型和框架解耦（计划书领域需求①~⑤）。
为什么做成"语言"而不是一个库?	预算是声明，不是过程：非程序员也能读写、能版本管理、能在执行前被检查——这些是库的函数调用做不到的。
AI 在项目里做了什么?	按计划书"人设计、AI 结对"：语言设计与关键决策由组员讨论确定，AI 生成了大部分代码和测试初稿；组员逐模块审阅（就是这份指南的用途），并以自动化测试验证。如实说明即可。
演示时间只有五分钟，怎么演?	建议顺序：缺省 7200（四种策略都触发，展开 chat_history 看摘要）→ 压缩场景 6000（低优先级被整块舍弃、规则报警告）→ 无解 1700（拒绝组装）→ 在源码里把 allergy_notes 的 overflow 改成 truncate_tail（约束⑨当场报错）。

第 6 章

在 GitHub 上审阅时的清单

审阅自己负责的模块时，逐条过一遍；有疑问就在对应代码行评论：

- ☐ 每个函数我都能用一句话说出它做什么
- ☐ 每个"设计决定"我都能说出理由，以及换一种做法的代价
- ☐ "动手验证"我都亲手跑过，现象和文中一致
- ☐ 能举出至少一个**测试没覆盖到**的输入（找到了就提 Issue，这是最有价值的审阅意见）
- ☐ 命名和注释是否清楚？有没有我读不懂的地方？
- ☐ 表格里的问题我能不看稿回答

附录 A

术语表

Token	词法分析的产物：(类型, 值, 行, 列)。类型有 KEYWORD / IDENT / INT / FLOAT / STRING / SYMBOL / EOF。
AST	抽象语法树：语法分析的产物，Program 下挂 WindowDecl、BlockDecl、RuleDecl，每个节点带行号。
IR / JSON 组装计划	AST 通过语义检查后翻译成的 JSON，引擎只读它；所有缺省值都已显式写出。
cap	块的额度上限（token），在源码里声明。
可用额度 B	本次组装能用的总额度，缺省 window × margin，调用方可以调小。
有效额度	压缩后某块本次实际可用的额度；B 充足时等于 cap，不足时按优先级削减。
margin	预留系数（缺省 0.9），window 的其余部分留给模型生成回答。
must_keep	死保优先级：不参与压缩；超出 cap 时拒绝组装，绝不静默裁剪。
综合属性 / 继承属性	属性文法概念：前者自底向上汇总（如 Σcap），后者自顶向下传递（如 window、margin）。
LL(1)	自顶向下分析时只看 1 个前瞻 Token 就能唯一确定用哪条产生式，无需回溯。
最大余数法	按比例分配整数名额：先发整数部分，剩余名额按小数部分从大到小补，总和精确。
BM25	经典的检索打分公式，综合词频、逆文档频率与文档长度。CBL 用它给检索段落排序。
快照测试	把完整输出存成文件，之后逐字比对，任何变化都需要人确认。
变异检查	故意改坏一处代码，确认测试会失败，用来证明测试真的在把关。
XSS	跨站脚本：不可信内容被当成 HTML/脚本执行。CBL 用 textContent 与 JSON 转义防范。

附录 B

命令速查

均在 `cbl/` 目录下执行。

`python tests/run_tests.py`	跑全部 54 个用例
`python -m cbl check examples/medical_assistant.cbl`	编译检查，输出 JSON 组装计划
`python -m cbl run examples/medical_assistant.cbl`	编译 + 组装，打印文本审计报告
`… run … --budget 6000`	指定可用额度 B
`… run … --query "提问"`	指定本轮用户提问（缺省读 materials/query.txt）
`… run … --out p.txt --json a.json --html a.html`	最终 Prompt / 审计 JSON / 独立 HTML 报告写盘
`python -m cbl serve`	预算实验台（Windows 可双击 启动预算实验台.bat）
`python -m cbl serve --lan`	局域网模式，手机同一 Wi-Fi 访问
`http://127.0.0.1:8000/?budget=6000`	实验台直达某个额度场景
`CBL_UPDATE_SNAPSHOTS=1 python tests/run_tests.py`	接受预期内的审计报告变化（PowerShell：先 $env:CBL_UPDATE_SNAPSHOTS=1）
`git checkout .`	动手实验改完代码后还原

附录 C

五分钟演示脚本

左列是建议的时间点，右列是可以直接照着讲的话。提前打开实验台，把 `?budget=7200`、`?budget=6000`、`?budget=1700` 存成书签。

时间	操作	讲解
0:00	**打开实验台（缺省 B = 7200）**	这是同一份 .cbl 声明：左边是源码，右边是这次组装的结果。四种裁剪策略都触发了。
0:40	**展开 chat_history 的「AI 实际看到的内容」**	8 轮对话，最近 4 轮保留原文，早期 4 轮压成了上面这段摘要——这就是 summarize。
1:20	**展开 retrieved_docs**	24 段检索资料按与提问的相关度排序，高血压、骨折这些无关段落被整段丢掉。
2:00	**点预设「压缩场景 6000」**	可用额度少了 1100：先压 low 级的工具输出，压到 0 整块舍弃；不够再压 medium 级的对话历史。high 与死保块纹丝不动。
2:40	**指向规则警告**	工具输出被饿死了，no_starvation 规则报警告——用户写的规则在运行期生效。
3:10	**点预设「无解 1700」**	额度低于死保总额 1800：引擎拒绝组装，没有任何内容送进模型。宁可报错，也不静默丢失过敏史。
3:50	**把 allergy_notes 的 overflow 改成 truncate_tail**	编译期当场报 E901：死保块不允许配置裁剪策略。错误在写下的那一刻就被发现。
4:30	**点「导出 HTML 审计报告」**	每次组装都能导出一份可存档的审计报告——AI 看到了什么、丢了什么、为什么，都可追溯。